

Source: <https://chatgpt.com/share/684995e4-aa14-800b-9ce5-f8c74cd34d9a>

This is a copy of a conversation between ChatGPT & Anonymous.

Report content

You said:

write a python script using youtube-dl to download songs from YouTube. It should only download the audio

ChatGPT said:

Sure! Here's a simple Python script that uses youtube-dl to download audio from YouTube videos:

python

CopyEdit

```
import youtube_dl
```

```
def download_audio(url):
```

```
    # Set the download options
```

```
    ydl_opts = {
```

```
        'format': 'bestaudio/best', # Get the best audio quality
```

```
        'postprocessors': [{
```

```
            'key': 'FFmpegExtractAudio', # Use ffmpeg to extract audio
```

```
            'preferredcodec': 'mp3', # Save as mp3
```

```
            'preferredquality': '192', # Set audio quality
```

```
        ]},
```

```
        'outtmpl': '%(title)s.%(ext)s', # Save file as video title
```

```
    }
```

```
    # Download the audio
```

```
    with youtube_dl.YoutubeDL(ydl_opts) as ydl:
```

```
        ydl.download([url])
```

```
if __name__ == '__main__':  
    video_url = input("Enter the YouTube URL: ")  
    download_audio(video_url)
```

How to Use

1. Install youtube-dl:

bash

CopyEdit

```
pip install youtube-dl
```

2. Install ffmpeg (needed for audio extraction):

- o macOS: brew install ffmpeg
- o Ubuntu: sudo apt install ffmpeg
- o Windows: Download from ffmpeg.org.

3. Save the script as download_audio.py.

4. Run it:

bash

CopyEdit

```
python download_audio.py
```

5. Enter the YouTube URL when prompted.

Let me know if you'd like me to expand this to handle playlists, different output formats, or GUI integration! 🚀 🎧

You said:

can you add a parameter to set download location

ChatGPT said:

Of course! Let's add an optional parameter to specify the download location. Here's the updated script:

python

CopyEdit

```
import youtube_dl
```

```
import os
```

```

def download_audio(url, download_path=''):
    # Set the download options
    ydl_opts = {
        'format': 'bestaudio/best', # Best audio quality
        'postprocessors': [{
            'key': 'FFmpegExtractAudio',
            'preferredcodec': 'mp3', # Save as mp3
            'preferredquality': '192', # Audio quality
        }],
        'outtmpl': os.path.join(download_path, '%(title)s.%(ext)s'), # Save to the specified directory
    }

    with youtube_dl.YoutubeDL(ydl_opts) as ydl:
        ydl.download([url])

if __name__ == '__main__':
    video_url = input("Enter the YouTube URL: ")
    download_dir = input("Enter download directory (leave blank for current directory): ").strip()

    # Use current directory if not specified
    if not download_dir:
        download_dir = '.'

    download_audio(video_url, download_dir)

```

Key Updates

- ✅ Added a `download_path` parameter to the `download_audio` function.
- ✅ The script prompts for the download directory, defaulting to the current directory if none is specified.
- ✅ Uses `os.path.join()` to ensure compatibility across operating systems.

Want to tweak it further, like adding error handling or supporting batch downloads? Let me know!



You said:

what about a parameter for the file name of the downloaded song

ChatGPT said:

Great idea! Let's add a parameter to specify the filename for the downloaded audio file. Here's the updated script:

python

CopyEdit

```
import youtube_dl
```

```
import os
```

```
def download_audio(url, download_path='', file_name=None):
```

```
    # If a custom file name is provided, set it as outtmpl
```

```
    if file_name:
```

```
        outtmpl = os.path.join(download_path, file_name + ':%(ext)s')
```

```
    else:
```

```
        outtmpl = os.path.join(download_path, '%(title)s.%(ext)s')
```

```
    # Set the download options
```

```
    ydl_opts = {
```

```
        'format': 'bestaudio/best',
```

```
        'postprocessors': [{
```

```
            'key': 'FFmpegExtractAudio',
```

```
            'preferredcodec': 'mp3',
```

```
            'preferredquality': '192',
```

```
        ]},
```

```
        'outtmpl': outtmpl,
```

```
    }
```

```
with youtube_dl.YoutubeDL(ydl_opts) as ydl:
```

```
    ydl.download([url])
```

```
if __name__ == '__main__':
```

```
    video_url = input("Enter the YouTube URL: ")
```

```
    download_dir = input("Enter download directory (leave blank for current directory): ").strip()
```

```
    file_name = input("Enter the desired file name (leave blank to use video title): ").strip()
```

```
    if not download_dir:
```

```
        download_dir = '.'
```

```
    if not file_name:
```

```
        file_name = None
```

```
    download_audio(video_url, download_dir, file_name)
```

Key Additions:

- ✓ Added an optional file_name parameter to specify the desired filename.
- ✓ If a filename is provided, it overrides the default video title name.
- ✓ Keeps file extension dynamic to match the audio format (mp3).

Let me know if you'd like to tweak this further — for instance, batch file downloads with custom names, or filename sanitization to ensure compatibility! 🎵

You said:

can you create a version of the script that downloads every video in a playlist?

ChatGPT said:

Sure! Let's tweak the script to support downloading audio from **every video in a playlist**.

Here's the updated version:

python

CopyEdit

```

import youtube_dl
import os

def download_playlist_audio(playlist_url, download_path='.', file_name_template=None):
    # Set the output template
    if file_name_template:
        # Use user-defined filename template
        outtmpl = os.path.join(download_path, file_name_template + '_%(playlist_index)s.%(ext)s')
    else:
        # Use video title by default
        outtmpl = os.path.join(download_path, '%(title)s.%(ext)s')

    # Configure youtube-dl options
    ydl_opts = {
        'format': 'bestaudio/best',
        'postprocessors': [{
            'key': 'FFmpegExtractAudio',
            'preferredcodec': 'mp3',
            'preferredquality': '192',
        }],
        'outtmpl': outtmpl,
        'yesplaylist': True, # Download entire playlist
        'ignoreerrors': True, # Skip errors for individual videos
    }

    with youtube_dl.YoutubeDL(ydl_opts) as ydl:
        ydl.download([playlist_url])

if __name__ == '__main__':

```

```

playlist_url = input("Enter the YouTube playlist URL: ")

download_dir = input("Enter download directory (leave blank for current directory): ").strip()

file_name_template = input(
    "Enter base file name template (optional). For example, 'my_playlist' will create files like
    'my_playlist_1.mp3', 'my_playlist_2.mp3', etc.: "
).strip()

if not download_dir:
    download_dir = '.'

if not file_name_template:
    file_name_template = None

download_playlist_audio(playlist_url, download_dir, file_name_template)

```

Key Features:

- ✅ Downloads every video in a playlist
- ✅ If a base file name is provided, filenames are indexed (e.g., my_playlist_1.mp3, my_playlist_2.mp3)
- ✅ Skips videos that might produce errors (ignoreerrors: True)
- ✅ Uses the playlist index (%(playlist_index)s) to keep files ordered

Want to add enhancements like parallel downloads, progress bars, or metadata embedding? Let me know! 🚀 🎧

You said:

getting this error on downloading playlist: ERROR: Unable to extract uploader id; please report this issue on <https://yt-dl.org/bug> . Make sure you are using the latest version; see <https://yt-dl.org/update> on how to update. Be sure to call youtube-dl with the --verbose flag and include its complete output.

ChatGPT said:

Ah, I see! This error (Unable to extract uploader id) typically happens because the youtube-dl tool is outdated and **YouTube changed their page structure**, breaking the older parsing logic.

🔧 How to Fix This

1. **Update to yt-dlp:**

youtube-dl is no longer actively maintained, so it's recommended to switch to its fork yt-dlp, which is much more up-to-date and fully compatible.

Install yt-dlp:

bash

CopyEdit

```
pip install -U yt-dlp
```

2. **Update the Script to Use yt-dlp:**

Just replace `import youtube_dl` with `import yt_dlp as youtube_dl` — the API is the same!

Updated Script with yt-dlp:

python

CopyEdit

```
import yt_dlp as youtube_dl
```

```
import os
```

```
def download_playlist_audio(playlist_url, download_path='.', file_name_template=None):
```

```
    if file_name_template:
```

```
        outtmpl = os.path.join(download_path, file_name_template + '_%(playlist_index)s.%(ext)s')
```

```
    else:
```

```
        outtmpl = os.path.join(download_path, '%(title)s.%(ext)s')
```

```
    ydl_opts = {
```

```
        'format': 'bestaudio/best',
```

```
        'postprocessors': [{
```

```
            'key': 'FFmpegExtractAudio',
```

```
            'preferredcodec': 'mp3',
```

```
            'preferredquality': '192',
```

```
        ]},
```

```
        'outtmpl': outtmpl,
```

```
        'yesplaylist': True,
```



```

        'ignoreerrors': True,
    }

    with youtube_dl.YoutubeDL(ydl_opts) as ydl:
        ydl.download([playlist_url])

if __name__ == '__main__':
    playlist_url = input("Enter the YouTube playlist URL: ")
    download_dir = input("Enter download directory (leave blank for current directory): ").strip()
    file_name_template = input(
        "Enter base file name template (optional). For example, 'my_playlist' will create files like
        'my_playlist_1.mp3', 'my_playlist_2.mp3', etc.: "
    ).strip()

    if not download_dir:
        download_dir = '.'

    if not file_name_template:
        file_name_template = None

    download_playlist_audio(playlist_url, download_dir, file_name_template)

```

✅ Summary

- The **error is due to youtube-dl being outdated**.
- **Switch to yt-dlp** for smoother downloads from YouTube (and other platforms).
- Let me know if you want me to help you install/update yt-dlp or modify the script to handle even more advanced options (like downloading subtitles, etc.). Happy downloading! 🎵 🎧

You said:

can you update single video script too

ChatGPT said:

Absolutely! Here's the updated **single-video download script** using yt-dlp instead of youtube-dl to avoid those extraction errors and keep everything fresh:

python

CopyEdit

```
import yt_dlp as youtube_dl
```

```
import os
```

```
def download_audio(url, download_path='', file_name=None):
```

```
    # Set the output template based on whether a custom filename is provided
```

```
    if file_name:
```

```
        outtmpl = os.path.join(download_path, file_name + '.*(ext)s')
```

```
    else:
```

```
        outtmpl = os.path.join(download_path, '%(title)s.%(ext)s')
```

```
    # Configure yt-dlp options
```

```
    ydl_opts = {
```

```
        'format': 'bestaudio/best',
```

```
        'postprocessors': [{
```

```
            'key': 'FFmpegExtractAudio',
```

```
            'preferredcodec': 'mp3',
```

```
            'preferredquality': '192',
```

```
        ]},
```

```
        'outtmpl': outtmpl,
```

```
    }
```

```
    with youtube_dl.YoutubeDL(ydl_opts) as ydl:
```

```
        ydl.download([url])
```

```
if __name__ == '__main__':  
    video_url = input("Enter the YouTube URL: ")  
    download_dir = input("Enter download directory (leave blank for current directory): ").strip()  
    file_name = input("Enter the desired file name (leave blank to use video title): ").strip()  
  
    if not download_dir:  
        download_dir = '.'  
  
    if not file_name:  
        file_name = None  
  
    download_audio(video_url, download_dir, file_name)
```

🔥 Key Changes:

- ✅ Uses **yt-dlp** instead of youtube-dl for better compatibility and stability.
- ✅ No other changes needed — everything else remains the same.

Want to extend it for batch downloads from a text file? Or add a progress bar? Let me know — we can build it together! 🚀 🎵

You said:

Can you create a version that takes a start and end timestamp from a video to convert to audio.

ChatGPT said:

Absolutely! You can use yt-dlp along with ffmpeg to **download a specific time segment** from a YouTube video and convert it to audio (e.g. MP3). This involves:

- Downloading the full video or audio stream.
- Trimming it using ffmpeg with the postprocessor_args option.

Here's the updated script:

✅ Download Audio Segment by Timestamps (Start/End)

python

CopyEdit

```
import yt_dlp as youtube_dl
```

```
import os
```

```
def download_audio_segment(url, start_time, end_time, download_path='', file_name=None):
```

```
    # Output filename
```

```
    if file_name:
```

```
        outtmpl = os.path.join(download_path, file_name + ':%(ext)s')
```

```
    else:
```

```
        outtmpl = os.path.join(download_path, '%(title)s.%(ext)s')
```

```
    # Construct ffmpeg arguments to trim audio
```

```
    ffmpeg_args = [
```

```
        '-ss', start_time, # start time
```

```
        '-to', end_time    # end time
```

```
    ]
```

```
    # yt-dlp options
```

```
    ydl_opts = {
```

```
        'format': 'bestaudio/best',
```

```
        'outtmpl': outtmpl,
```

```
        'postprocessors': [{
```

```
            'key': 'FFmpegExtractAudio',
```

```
            'preferredcodec': 'mp3',
```

```
            'preferredquality': '192',
```

```
        ]},
```

```
        'postprocessor_args': ffmpeg_args,
```

```
        'prefer_ffmpeg': True,
```

```
    }
```

```
with youtube_dl.YoutubeDL(ydl_opts) as ydl:
```

```
    ydl.download([url])
```

```
if __name__ == '__main__':
```

```
    video_url = input("Enter the YouTube URL: ").strip()
```

```
    start_time = input("Enter start time (e.g., 00:01:00): ").strip()
```

```
    end_time = input("Enter end time (e.g., 00:02:30): ").strip()
```

```
    download_dir = input("Enter download directory (leave blank for current directory): ").strip()
```

```
    file_name = input("Enter output file name (without extension): ").strip()
```

```
    if not download_dir:
```

```
        download_dir = '.'
```

```
    if not file_name:
```

```
        file_name = None
```

```
    download_audio_segment(video_url, start_time, end_time, download_dir, file_name)
```

✂ Requirements:

- yt-dlp:

bash

CopyEdit

pip install -U yt-dlp

- ffmpeg (must be installed and in PATH):
 - macOS: brew install ffmpeg
 - Ubuntu: sudo apt install ffmpeg
 - Windows: Download from ffmpeg.org
-

Notes:

- start_time and end_time must be in hh:mm:ss format.
- The downloaded segment will be saved in MP3 format.
- If file_name is omitted, it uses the original video title.

Want to extract multiple segments from the same video? Just let me know — we can automate that too. 🎧 🔧

You said:

I received this error: ERROR: [youtube] uTVrgz9OGg0: Sign in to confirm you're not a bot. Use --cookies-from-browser or --cookies for the authentication. See <https://github.com/yt-dlp/yt-dlp/wiki/FAQ#how-do-i-pass-cookies-to-yt-dlp> for how to manually pass cookies. Also see <https://github.com/yt-dlp/yt-dlp/wiki/Extractors#exporting-youtube-cookies> for tips on effectively exporting YouTube cookies

ChatGPT said:

This error means **YouTube is blocking the request** because it's detecting automated traffic and asking for a login to confirm you're not a bot. It's a common issue when:

- The video is **age-restricted**.
- The video requires **login authentication**.
- YouTube is being unusually aggressive with **bot detection**.

✅ **Solution: Use Browser Cookies with yt-dlp**

You can pass your YouTube login session cookies from your browser to yt-dlp, so it acts like **you** when downloading.

Step-by-Step Fix

✅ **1. Install yt-dlp (if not already):**

bash

CopyEdit

```
pip install -U yt-dlp
```

✅ **2. Install browser-cookie3 (to automatically read browser cookies):**

bash

CopyEdit

pip install browser-cookie3

✅ 3. Modify your Python script to use browser cookies:

Add this import:

python

CopyEdit

```
import browser_cookie3
```

Update your ydl_opts like this:

python

CopyEdit

```
cookies = browser_cookie3.load() # Load cookies from your default browser
```

```
ydl_opts = {
    'format': 'bestaudio/best',
    'outtmpl': 'outtmpl',
    'postprocessors': [{
        'key': 'FFmpegExtractAudio',
        'preferredcodec': 'mp3',
        'preferredquality': '192',
    }],
    'postprocessor_args': ffmpeg_args,
    'prefer_ffmpeg': True,
    'cookiefile': None, # Not needed when using browser-cookie3
    'nocheckcertificate': True,
    'quiet': False,
    'cookiesfrombrowser': ('chrome',),
}
```

But **yt-dlp itself doesn't use cookiesfrombrowser directly** in the Python API. Instead, you can pass cookies manually:

✅ 4. Better: Export cookies to a file (recommended and reliable)

1. Install **EditThisCookie** browser extension.
2. Log into YouTube in your browser.
3. Export cookies from youtube.com as a .txt file.
4. Save it as cookies.txt in the same directory as your script.

Update your script like this:

python

CopyEdit

```
ydl_opts = {  
    'format': 'bestaudio/best',  
    'outtmpl': outtmpl,  
    'postprocessors': [{  
        'key': 'FFmpegExtractAudio',  
        'preferredcodec': 'mp3',  
        'preferredquality': '192',  
    }],  
    'postprocessor_args': ffmpeg_args,  
    'prefer_ffmpeg': True,  
    'cookiefile': 'cookies.txt', # Point to your cookie file  
}
```

Summary

Fix

What it Does

Export browser cookies Bypasses bot check and allows age-restricted/private video downloads

Use cookies.txt Most stable and recommended

Use browser-cookie3 Less manual but may not work reliably on all platforms/browsers

Let me know if you'd like help automating the cookie export or if you want a more secure, app-specific solution.

